# Clinical Data Visualizer – User Guide

Clinical Data Visualizer Team

February 18, 2026

## Contents

# 1 Introduction

Clinical Data Visualizer is an interactive dashboard for visualizing clinical physiological signals. It allows clinicians and researchers to explore, compare, and annotate time-series data from multiple medical devices in a single unified interface.

## 1.1 Key Features

- **Multi-source visualization**: Display signals from up to 9 different clinical data sources simultaneously.
- **Interactive plots**: Zoom, pan, and explore data at any time scale with automatic resampling.
- **Annotations**: Draw lines and rectangles directly on plots to mark events or regions of interest. Annotations are saved and persist across sessions.
- **Flexible configuration**: Choose which signals to display, customize labels, units, colors, and group related signals together.
- **Export**: Generate standalone HTML visualizations for sharing.

## 1.2 Supported Data Sources

| Data Source | Description |
|---|---|
| Philips Waves | High-frequency waveform data from Philips patient monitors |
| Philips Numerics | Numeric parameter data from Philips monitors (heart rate, SpO2, etc.) |
| EIT (PulmoVista) | Electrical Impedance Tomography data from Draeger PulmoVista |
| FluxMed Signals | Waveform data from FluxMed respiratory monitors |
| FluxMed Parameters | Parameter data from FluxMed monitors |
| Servo-U | Ventilator data from Getinge Servo-U |
| Mindray | Patient monitor data from Mindray scopes |
| Syringe | Syringe pump infusion data |
| Other (Generic) | Auto-discovers CSV or Parquet files with datetime columns |

# 2 Launching the Application

## 2.1 Starting the App

Locate the **ClinicalVisuAppAlexis** executable in the application folder and double-click it.

A terminal window will appear showing the application starting up. After a few seconds, your default web browser will automatically open at:

`http://127.0.0.1:8050`

If the browser does not open automatically, manually navigate to the address above.

Application launch screen

Figure 1: Application launch screen

## 2.2 Application Overview

The interface is organized top-to-bottom in the following order:

1. **Database Options** – Load or select a visualization configuration.
2. **Patient Options** – Configure data folder, time range, and per-source settings.
3. **Process Button** – Start the visualization.
4. **Shape Controls** – Manage annotations (visible after processing).
5. **Visualization Area** – Interactive plots.

Application main interface

Figure 2: Application main interface

# 3 Preparing Your Data

## 3.1 Patient Folder Structure

Each patient's data must be organized in a root folder with **one subfolder per data source**. The application automatically identifies data sources based on keywords in subfolder names.

```
Patient1/
  philips_waves/         Philips waveform data (.parquet)
  philips_numerics/      Philips numeric data
  eit/                   EIT PulmoVista data (.asc)
  fluxmed_signals/       FluxMed waveform data
  fluxmed_parameters/    FluxMed parameter data
  servo_u/               Servo-U ventilator data (.sta)
  mindray/               Mindray scope data (.xml or .csv)
  syringe/               Syringe pump data
  other/                 Generic data (.csv or .parquet)
  tdv_visu/              Auto-created: cached data and outputs
```

You only need subfolders for the data sources you actually have. Empty or missing subfolders are simply skipped.

## 3.2 Folder Naming Rules

Folder names are **flexible** – they just need to contain the required keywords (case-insensitive, any separator allowed):

| Data Source | Required Keywords | Examples |
|---|---|---|
| Philips Waves | `philips` + `waves` | `philips_waves`, `Philips-Waves`, `waves_philips` |
| Philips Numerics | `philips` + `numerics` | `philips_numerics`, `Philips-Numerics` |
| EIT | `eit` | `eit`, `EIT`, `EIT_Data` |
| FluxMed Signals | `fluxmed` + `signals` | `fluxmed_signals`, `FluxMed-Signals` |
| FluxMed Parameters | `fluxmed` + `parameters` | `fluxmed_parameters`, `FluxMed_Parameters` |
| Servo-U | `servo` | `servo_u`, `Servo-U`, `SERVO` |
| Mindray | `mindray` | `mindray`, `Mindray` |
| Syringe | `syringe` | `syringe`, `Syringe`, `syringe_pumps` |
| Other | `other` | `other`, `Other` |

## 3.3 Expected File Types

- **Philips Waves**: `.parquet` files
- **Philips Numerics**: Files containing "numerics" in the filename
- **EIT**: `.asc` files
- **FluxMed**: Files containing "signals" or "parameters" in the filename
- **Servo-U**: `.sta` files
- **Mindray**: `.xml` or `.csv` files
- **Syringe**: Files containing "syringe" in the filename
- **Other**: `.csv` or `.parquet` files (auto-discovers columns with datetime values)

Patient folder structure example

Figure 3: Patient folder structure example

# 4 Loading Database Options

Database options define **which data sources to enable** and **how signals should be displayed** (labels, units, colors, grouping).

## 4.1 Option 1: Default Visualization (Quick Start)

Click the green **"Default visualization (all sources)"** button. This automatically enables all 9 data sources with their default display settings. No configuration file is needed.

This is the recommended starting point for new users.

Default visualization button

Figure 4: Default visualization button

## 4.2 Option 2: Custom Configuration File

Click the blue **"Upload config file"** button to load a custom `database_options.json` file. This gives you full control over which sources are enabled and how each signal is displayed.

See Section 9 for a detailed description of the configuration file format.

Upload config file button

Figure 5: Upload config file button

# 5 Configuring Patient Options

After loading database options, the **Patient Options** form appears. It is divided into two parts: global options and per-source options.

## 5.1 Global Options

These apply to all data sources:

| Option | Description |
|---|---|
| **Path to data (folder)** | Full path to the patient's root data folder |
| **Time start filter** | Start of the time window to display (format: `YYYY-MM-DD HH:MM:SS`). Leave empty to use all available data. |
| **Time end filter** | End of the time window to display. Leave empty to use all available data. |
| **Re-use data if already loaded once** | When checked, reuses previously cached `.parquet` files from the `tdv_visu/` folder, significantly speeding up subsequent loads. |

Global patient options

Figure 6: Global patient options

## 5.2 Per-Source Options

Below the global options, each enabled data source may have additional settings displayed in individual cards arranged in a two-column grid. Common per-source options include:

- **Time shift** (seconds): Adjust the time alignment of a source relative to others. Useful when devices were not perfectly synchronized.
- **Day**: Specify the recording date for sources that require it (e.g., EIT data).

Only data sources present in the loaded database options will show their configuration cards.

Per-source options

Figure 7: Per-source options

# 6 Processing the Visualization

Once you have configured the patient options, click the large orange **"Process visualization"** button to start generating the plots.

## 6.1 What Happens During Processing

1. **Validation**: The application verifies that all mandatory fields are filled in and that the data folder exists.
2. **Data Discovery**: For each enabled data source, the application scans the patient folder for matching subfolders and files.
3. **Data Loading**: Raw data files are parsed according to each source's format.
4. **Formatting**: Signals are filtered, resampled, and converted using your database options (labels, units, time range).
5. **Caching**: Processed data is saved as `.parquet` files in the `tdv_visu/` subfolder for faster reloading next time.
6. **Plot Generation**: Interactive Plotly figures are created and displayed in the visualization area.

A success message appears when processing completes. If no data is found for a source, it is silently skipped.

Processing visualization

Figure 8: Processing visualization

# 7 Interacting with Plots

## 7.1 Navigation Controls

Each plot provides a toolbar (top-right corner) with the following tools:

| Tool | Action |
|---|---|
| **Zoom** | Click and drag to zoom into a rectangular region |
| **Pan** | Click and drag to move the view |
| **Zoom In / Zoom Out** | Incremental zoom buttons |
| **Autoscale** | Reset the view to fit all data |
| **Reset Axes** | Return to the original view |
| **Download as PNG** | Save the current plot view as an image |

## 7.2 Zooming and Panning

- **Scroll wheel**: Zoom in/out on the x-axis.
- **Click and drag**: Select a region to zoom into.
- **Double-click**: Reset the axes to show all data.

## 7.3 Dynamic Resampling (FigureResampler)

For high-frequency signals (e.g., waveforms sampled at hundreds of Hz), the application uses **Plotly-Resampler** to dynamically load detail as you zoom in. When viewing a long time range, the plot shows a downsampled overview. As you zoom into a shorter time window, the full-resolution data is loaded automatically.

This keeps the interface responsive even with millions of data points.

Interactive plot navigation

Figure 9: Interactive plot navigation

# 8 Annotations and Shape Tools

The annotation system lets you mark events, time periods, or regions of interest directly on the plots. Annotations are saved to an `annotations.json` file in the `tdv_visu/` folder and persist across sessions.

## 8.1 Drawing Annotations

Use the Plotly drawing tools in each plot's toolbar:

- **Draw Line**: Click two points to draw a vertical line marking a specific event.
- **Draw Rectangle**: Click and drag to highlight a time region or value range.

Each shape can be given a name and a color via the edit popup that appears after drawing.

## 8.2 Shape Management

After processing, the **Shape Controls** section becomes visible below the Process button:

- **Shape Dropdown**: Lists all annotations across all figures, showing the figure name and shape label.
- **Modify Button**: Opens the edit popup for the selected annotation (change name, color, or whether it spans all subplots).
- **Delete Button**: Removes the selected annotation.

## 8.3 Annotation Properties

Each annotation has the following properties:

- **Name**: A text label displayed on the shape.
- **Color**: The line or fill color.
- **Global**: When enabled, the shape spans all subplots in the figure (using paper y-coordinates).

## 8.4 Persistence

Annotations are automatically saved to `annotations.json` in the patient's `tdv_visu/` folder whenever you create, modify, or delete a shape. They are reloaded when you re-process the same patient data.

Annotation tools and shape management

Figure 10: Annotation tools and shape management

# 9 Configuration File Reference

## 9.1 patient_options.json

This file defines patient-specific settings.

```json
{
    "data_folder": "/path/to/patient/data",
    "datetime_start": "2024-10-08 10:00:00",
    "datetime_end": "2024-10-08 12:00:00",
    "quick_load": false,
    "philips_waves": {
        "time_shift": 20.0
    },
    "eit": {
        "day": "2024-10-08"
    }
}
```

| Key | Type | Description |
|---|---|---|
| `data_folder` | string | Path to the patient's root data folder |
| `datetime_start` | string or null | Start of the time window (`YYYY-MM-DD HH:MM:SS`) |
| `datetime_end` | string or null | End of the time window |
| `quick_load` | boolean | Reuse cached parquet files |
| `<source_name>` | object | Per-source options (e.g., `time_shift`, `day`) |

## 9.2 database_options.json

This file controls which data sources are active and how signals are displayed.

### 9.2.1 Top-Level Structure

```json
{
    "global": {
        "grouped_fields": { "Group Name": ["signal1", "signal2"] }
    },
    "philips_waves": { ... },
    "philips_numerics": { ... },
    "eit": { ... }
}
```

Each data source key is optional – only include the sources you want to enable.

### 9.2.2 Per-Source Fields

| Key | Description |
|---|---|
| `field_display` | Array of signal names to show (others are hidden) |
| `data.label_correspondence` | Map raw signal names to display labels |
| `data.unit_conversion` | Multiplication factors for unit conversion |
| `data.unit_info` | Display unit strings (e.g., "mmHg", "cmH2O") |

| Key | Description |
|---|---|
| `data.unit_range` | Fixed Y-axis ranges as `[min, max]` |
| `data.color` | Custom colors per signal |
| `data.priority` | Plot ordering priority (lower = higher on page) |
| `data.period_resampling` | Resampling period in seconds |
| `grouped_fields` | Group signals onto the same subplot |
| `loop` | Define PV-loop plots (e.g., `{"pv_loop": ["Pressure", "Volume"]}`) |
| `numerics.period_resampling` | Resampling for numeric parameters |
| `numerics.priority` | Plot priority for numerics |

#### 9.2.3 Global Fields

| Key | Description |
|---|---|
| `global.grouped_fields` | Group signals from different data sources onto the same subplot |

See `example/option_files/` in the source repository for complete example files.

# 10 Troubleshooting

## 10.1 Browser Does Not Open Automatically

If the browser does not open after launching the application, manually navigate to:

`http://127.0.0.1:8050`

Ensure no other application is using port 8050. If needed, close the terminal window which was opened in the app and restart the application.

## 10.2 No Data Found

If the visualization is empty or a data source shows no signals:

- Verify that the **data folder path** is correct and accessible.
- Check that subfolders follow the **naming conventions** (see Section 3).
- Ensure the subfolder contains files in the **expected format** for that data source.
- Check that the data source is **enabled** in your database options (or use "Default visualization" to enable all).

## 10.3 Slow Loading

Large datasets may take time to load on the first run. To speed up subsequent loads:

- Enable the **"Re-use data if already loaded once"** (quick_load) option. This uses the cached `.parquet` files in `tdv_visu/` instead of re-reading raw data files.

## 10.4 Time Alignment Issues

If signals from different sources appear misaligned in time:

- Use the **Time shift** option in the per-source settings to adjust alignment.
- Verify that the correct **day** or **date** is set for sources that require it (e.g., EIT).

## 10.5 Application Crashes or Errors

- Check the terminal window for error messages.
- Log files are available in the `logs/` directory (if running from source).
- Ensure the data files are not corrupted or truncated.

# 11 Appendix: Supported Data Sources

| Source | Module Name | Keywords | File Types | Typical Signals |
|---|---|---|---|---|
| Philips Waves | `philips_waves` | `philips`, `waves` | `.parquet` | ART, PAP, CO2, respiratory pressure/volume |
| Philips Numerics | `philips_numerics` | `philips`, `numerics` | numerics in name | Heart rate, SpO2, FiO2, blood pressure |
| EIT | `eit` | `eit` | `.asc` | Global/local impedance, impedance percentages |
| FluxMed Signals | `fluxmed_signals` | `fluxmed`, `signals` | signals in name | Respiratory waveforms |
| FluxMed Parameters | `fluxmed_parameters` | `fluxmed`, `parameters` | parameters in name | Respiratory parameters |
| Servo-U | `servo_u` | `servo` | `.sta` | Ventilator waveforms and settings |
| Mindray | `mindray` | `mindray` | `.xml`, `.csv` | Monitor waveforms and parameters |
| Syringe | `syringe` | `syringe` | syringe in name | Infusion rates and volumes |
| Other | `other` | `other` | `.csv`, `.parquet` | Any time-series with datetime columns |